

Problem Max Flow

Ewa Kasprzak

21 stycznia 2025

1 Problem Maksymalnego Przepływu

Opis Problemu

1. **Model sieci:** Sieć $G = (N, A)$, gdzie:
 - N to zbiór wierzchołków,
 - A to zbiór łuków,
 - Każdy łuk $(i, j) \in A$ ma nieujemną pojemność u_{ij} .
2. **Wyróżnione węzły:** Sieć ma dwa specjalne węzły:
 - s - źródło,
 - t - ujście.
3. **Zadanie:** Znaleźnienie maksymalnego przepływu ze źródła s do ujścia t przy uwzględnieniu pojemności łuków i zachowaniu bilansu masy we wszystkich węzłach.
4. **Sformalizowanie problemu:**
 - **Maksymalizacja:** v
 - **Warunki:** Dla każdego $(i, j) \in A$.
$$\sum_{j:(i,j) \in A} X_{ij} - \sum_{j:(j,i) \in A} X_{ji} = \begin{cases} v & \text{dla } i = s \\ 0 & \text{dla wszystkich } i \in N \setminus \{s, t\} \\ -v & \text{dla } i = t \end{cases}$$
5. **Sieć rezydualna:** Sieć, w której można przesyłać dodatkowy przepływ, obliczana jako $r_{ij} = u_{ij} - X_{ij} + X_{ji}$.
6. **Przekrój $s-t$:** Podział wierzchołków na dwie grupy, gdzie s jest w jednej grupie, a t w drugiej. Pojemność przekroju to suma pojemności łuków wychodzących z jednej grupy do drugiej.
7. **Twierdzenie max-flow min-cut:** Wartość maksymalnego przepływu jest równa pojemności minimalnego przekroju $s-t$.

2 Algorytm Forda - Fulkersona, implementacja Edmondsa - Karpa

Implementacja algorytmu Edmondsa-Karpa, będąca odmianą algorytmu Forda-Fulkersona, polega na znalezieniu maksymalnego przepływu w sieci przepływowej przy użyciu przeszukiwania w szerz (BFS) do znajdowania ścieżek powiększających. Poniżej znajduje się pseudokod omawianego algorytmu:

```

algorithm labeling;
begin
  label node  $t$ ;
  while  $t$  is labeled do
    begin
      unlabel all nodes;
      set  $\text{pred}(j) := 0$  for each  $j \in N$ ;
      label node  $s$  and set  $\text{LIST} := \{s\}$ ;
      while  $\text{LIST} \neq \emptyset$  or  $t$  is unlabeled do
        begin
          remove a node  $i$  from  $\text{LIST}$ ;
          for each arc  $(i, j)$  in the residual network emanating from node  $i$  do
            if  $r_{ij} > 0$  and node  $j$  is unlabeled then set  $\text{pred}(j) := i$ , label node  $j$ , and
              add  $j$  to  $\text{LIST}$ ;
          end;
          if  $t$  is labeled then augment
        end;
      end;
    end;

  procedure augment;
  begin
    use the predecessor labels to trace back from the sink to the source to
      obtain an augmenting path  $P$  from node  $s$  to node  $t$ ;
     $\delta := \min\{r_{ij} : (i, j) \in P\}$ ;
    augment  $\delta$  units of flow along  $P$  and update the residual capacities;
  end;

```

Rysunek 1: Pseudokod algorytmu Edmondsa - Karpa. Źródło: [1]

Złożoność przedstawionego algorytmu wynosi $O(nm^2)$, gdzie n to liczba wierzchołków, a m to liczba krawędzi w grafie:

- Wyznaczenie najkrótszej ścieżki zajmuje $O(m)$.
- Wybierając najkrótsze ścieżki wykonamy co najwyżej $O(nm)$ powiększeń przepływu

3 Algorytm Shortest Augmenting Path

Ze względu na to, że istnieją sieci, które wymagają $\Omega(nm)$ powiększeń przepływu polem do optymalizacji jest złożoność operacji wykonywanych na każde powiększenie. Celem jest zejście ze złożoności $O(m)$ do złożoności $O(n)$. Odpowiedzią na to zagadnienie jest algorytm Shortest Augmenting Path.

W tym celu wprowadzamy funkcję $d[i]$, gdzie $i \in N$, która wyznacza długość najkrótszej ścieżki od wierzchołka i do ujścia t w badanej sieci residualnej. Mamy więc:

$$d(t) = 0$$

$$d(i) \leq d(j) + 1$$

dla każdego łuku $(i, j) \in A$. Łukiem dopuszczalnym nazywamy $(i, j) \in A$ taki, że

$$d(i) = d(j) + 1$$

Ideą algorytmu Shortest Augmenting Path będzie konstruowanie ścieżek dopuszczalnych, czyli takich, dla których każdy łuk jest dopuszczalny. Taka ścieżka jest równocześnie najkrótszą ścieżką powiększającą z s do t .

Istotną obserwacją, która pozwala na zmniejszenie złożoności do $O(n)$ jest fakt, że odległość dowolnego wierzchołka od ujścia po kolejnych powiększeniach jest niemalejąca. Pozwala to na ustalenie dokładnych wartości $d[i]$ na początku działania algorytmu, konstruowanie dopuszczalnych ścieżek w każdej iteracji oraz ewentualną korektę odległości wierzchołków od źródła.

- **advance(i)** to funkcja, która dla wierzchołka $i \in N$ znajduje dopuszczalny łuk lub informuje o braku takowego
- **retreat(i)** to funkcja, która aktualizuje odległość wierzchołka i od ujścia w przypadku, gdy nie ma on żadnego dozwolonego łuku w sieci residualnej. Nowa odległość jest ustawiana jako $\min\{d[j] + 1 : (i, j) \in A\}$.
- **augment()** to funkcja wykonująca powiększenie przepływu po wyznaczonej przez ciąg operacji **advance(i)** ścieżce minimalną pojemnością łuku na tej ścieżce

Algorytm kończy się, gdy $d[s] \geq n$. Pseudokod algorytmu Shortest Augmenting Path został przedstawiony na rysunku 2

```

algorithm shortest augmenting path;
begin
   $x := 0$ ;
  obtain the exact distance labels  $d(i)$ ;
   $i := s$ ;
  while  $d(s) < n$  do
    begin
      if  $i$  has an admissible arc then
        begin
          advance( $i$ );
          if  $i = t$  then augment and set  $i = s$ 
        end
      else retreat( $i$ )
    end;
  end;

```

Figure 7.5 Shortest augmenting path algorithm.

```

procedure advance( $i$ );
begin
  let  $(i, j)$  be an admissible arc in  $A(i)$ ;
   $\text{pred}(j) := i$  and  $i := j$ ;
end;

```

(a)

```

procedure retreat( $i$ );
begin
   $d(i) := \min\{d(j) + 1 : (i, j) \in A(i) \text{ and } r_{ij} > 0\}$ ;
  if  $i \neq s$  then  $i := \text{pred}(i)$ ;
end;

```

(b)

```

procedure augment;
begin
  using the predecessor indices identify an augmenting
  path  $P$  from the source to the sink;
   $\delta := \min\{r_{ij} : (i, j) \in P\}$ ;
  augment  $\delta$  units of flow along path  $P$ ;
end;

```

(c)

Figure 7.6 Procedures of the shortest augmenting path algorithm.

Rysunek 2: Pseudokod algorytmu Shortest Augmenting Path. Źródło: [1]

Algorytm można dodatkowo ulepszyć poprzez przechowywanie w każdym z wierzchołków wskaźnika na ostatnio przeglądane sąsiada oraz rozpoczęcia poszukiwania dopuszczalnego łuku od kolejnego z sąsiadów.

Kolejnym z ulepszeń jest przechowywanie tablicy $\text{numb}[0..n-1]$, gdzie $\text{numb}(k)$ oznacza liczbę węzłów o etykiecie $d[i] = k$. Początkowo tablica jest zainicjowana podczas obliczania dokładnych etykiet. Na początku wartości nieze-

rowe występują kolejno po sobie, np. $\text{numb}(1) = n-1$, $\text{numb}(2) = 1$, $\text{numb}(3) = 0$, a z czasem może się zmieniać, np. $\text{numb}(1) = 2$, $\text{numb}(2) = n-2$. Wartość $\text{numb}(0)$ jest zawsze równa 1 (odległość ujścia od siebie samego). Jeśli $\text{numb}(k) = 0$, a dla $j > k$ $\text{numb}(j) > 0$, algorytm kończy działanie. To usprawnienie pozwala algorytmowi efektywnie wykrywać istnienie przekroju o pojemności równej wielkości przepływu, często na długo przed osiągnięciem warunku $d(s) \geq n$, co sygnalizuje zakończenie algorytmu zgodnie z twierdzeniem o maksymalnym przepływie i minimalnym przekroju.

4 Skojarzenie w grafie dwudzielnym

Aby wykorzystać algorytm Edmondsa-Karpa do wyznaczenia skojarzenia o największej liczbie elementów w grafie dwudzielnym, należy najpierw przekształcić problem skojarzenia w problem maksymalnego przepływu. Proces ten obejmuje następujące kroki:

1. Modelujemy graf dwudzielną $G = (X_1, X_2, E)$, gdzie X_1 i X_2 to dwa rozłączne zbiory wierzchołków, a E to zbiór krawędzi o pojemności 1 łączących wierzchołki z X_1 i X_2 .
2. Tworzymy sieć przepływową $G' = (V', E')$, gdzie:
 - V' zawiera wszystkie wierzchołki z X_1 i X_2 , oraz dwa dodatkowe wierzchołki: źródło s i ujście t .
 - E' zawiera wszystkie krawędzie z E , oraz:
 - Krawędzie o pojemności 1 z s do każdego wierzchołka w X_1 .
 - Krawędzie o pojemności 1 z każdego wierzchołka w X_2 do t .

Po skonstruowaniu sieci przepływowej G' , możemy zastosować algorytm Edmondsa-Karpa, aby znaleźć maksymalny przepływ. Algorytm ten, wykorzystując przeszukiwanie wszerz (BFS) do znajdowania ścieżek powiększających, pozwala na efektywne obliczenie maksymalnego przepływu w sieci.

Maksymalny przepływ w sieci G' jest równy wielkości największego skojarzenia w oryginalnym grafie dwudzielnym G . W ten sposób, po znalezieniu maksymalnego przepływu, możemy odczytać skojarzenia poprzez analizę krawędzi wybranych w ramach ścieżek powiększających.

Wykorzystując do wyznaczenia największego skojarzenia w grafie dwudzielnym implementację Edmondsa-Karpa czas działania tego programu jest równy $O(nm^2)$.

5 Problem Największego Przepływu Jako Problem Programowania Liniowego

Problem maksymalnego przepływu można sformułować jako problem programowania liniowego, w którym celem jest maksymalizacja całkowitego przepływu ze źródła do ujścia w sieci przepływowej.

Zmienne decyzyjne

Zmiennymi decyzyjnymi są f_{ij} dla każdego łuku (i, j) w sieci oznaczające wyznaczony przepływ po danym łuku w rozwiązaniu optymalnym.

Funkcja celu

W problemie maksymalnego przepływu dążymy do maksymalizacji całkowitego przepływu z źródła s . Funkcja celu jest wyrażona jako suma przepływów wychodzących z s :

$$\max \sum_{v: (s,v) \in A} f_{sv}$$

Ograniczenia

Aby zapewnić poprawność przepływu w sieci, musimy uwzględnić następujące ograniczenia:

- Przepływ przez każdą krawędź nie może przekraczać jej pojemności:

$$f_{ij} \leq u_{ij} \quad \forall (i, j) \in A$$

- Suma przepływów wchodzących i wychodzących z każdego wierzchołka (poza źródłem i ujściem) musi się równoważyć:

$$\sum_u f_{uv} - \sum_w f_{vw} = 0 \quad v \in N \setminus \{s, t\}$$

- Dla wszystkich zmiennych przepływu musimy mieć warunek nieujemności:

$$f_{ij} \geq 0 \quad \forall (i, j) \in A$$

6 Wyznaczanie Maksymalnego Przepływu w Hyperkostce

Dla k -elementowego ciągu bitowego x definiujemy wagę Hamminga $H(x)$ jako liczbę jedynek w tym ciągu (waga Hamminga H przyjmuje wartości ze zbioru

$\{0, 1, \dots, k\}$). Niech ponadto $Z(x)$ będzie liczbą zer w ciągu x .

Dla danego $k \in \mathbb{N}$ rozważmy k -wymiarową skierowaną hiperkostkę $H_k = (N_k, A_k)$, tj. graf skierowany o $|N_k| = 2^k$ wierzchołkach, których etykietami są różne ciągi binarne długości k . Krawędzie (łuki) w hiperkostce H_k łączą wierzchołki etykietowane ciągami różniącymi się na dokładnie jednej pozycji i prowadzą od wierzchołka z etykietą o mniejszej wadze Hamminga do wierzchołka z etykietą o większej wadze. Łatwo zauważyć, że każdy wierzchołek $i \in N_k$ jest początkiem lub końcem dokładnie k łuków, a k -bitowe ciągi etykietujące wierzchołki mogą być traktowane jako binarne reprezentacje liczb naturalnych od 0 do $2^k - 1$.

Stąd formalnie hiperkostkę skierowaną możemy zdefiniować jako graf o wierzchołkach $N_k = \{0, 1, \dots, 2^k - 1\}$, gdzie etykieta $e(i)$ wierzchołka i jest k -bitową reprezentacją liczby i , a zbiorem krawędzi (łuków) jest $A_k = \{(i, j) \in N_k \times N_k : e(j) - e(i) \geq 0 \wedge H(e(j) - e(i)) = 1\}$ (w definicji zbioru krawędzi A_k wyrażenie $e(j) - e(i)$ oznacza różnicę wektorów z $\{0, 1\}^k$ reprezentujących etykiety wierzchołków j oraz i). Dla tak zdefiniowanego grafu H_k mamy $|A_k| = k \cdot 2^{k-1}$. Pojemność u_{ij} każdej krawędzi $(i, j) \in A_k$ losujemy niezależnie z rozkładem jednostajnym ze zbioru $\{1, \dots, 2^l\}$, gdzie $l = \max\{H(e(i)), Z(e(i)), H(e(j)), Z(e(j))\}$.

W celu przetestowania algorytmów z zakresu problemu max flow na opisanym grafie zaimplementowano w języku Julia odpowiednią strukturę grafu oraz dostosowano algorytmy w następujący sposób.

Implementacja - max flow

- Graf reprezentowany jako zbiór wierzchołków

```
mutable struct Vertex
    value::Int64
    label::Vector{Int64}
    h::Int64
    labeled::Bool
    num_of_edges::Int64
    neighbours::Vector{Tuple{Vertex, Int64, Int64}}
    last_visited::Int64
    d::Int64
end
```

- Generowanie krawędzi

```
function generate_edges(vertices::Vector{Vertex}, k::Int64)::Int64
    edge_count = 0
    for i in 1:length(vertices)
        seed = rand(1:typemax(UInt64))
        rng = MersenneTwister(seed)
```

```

    v = vertices[i]
    for j in (i+1):length(vertices)
        w = vertices[j]
        if sum(v.label .!= w.label) == 1
            l = max(v.h, k - v.h, w.h, k - w.h)
            capacity = rand(rng, 1:(2^l))
            push!(v.neighbours, (w, capacity, 0))
            push!(w.neighbours, (v, 0, 0))
            v.num_of_edges += 1
            w.num_of_edges += 1
            edge_count += 1
        end
    end
end
return edge_count
end

```

- Algorytm Edmondsa Karpa

```

function edmonds_karp(vertices::Vector{Vertex}, start::Vertex, finish::Vertex)
::Tuple{Int64, Int64}
    max_flow = 0
    found_paths = 0

    finish.labeled = true
    while finish.labeled
        list = Queue{Vertex}()
        pred = Dict{Vertex, Vertex}()

        for v in vertices
            v.labeled = false
        end

        enqueue!(list, start)
        start.labeled = true

        while !isempty(list) && !finish.labeled
            v = dequeue!(list)
            for (w, capacity) in v.neighbours
                if capacity > 0 && !w.labeled
                    w.labeled = true
                    pred[w] = v
                    enqueue!(list, w)
                end
            end
        end
    end
end

```



```

        if finish.labeled

            path = reconstruct_path(pred, start, finish)
            min_capacity = find_min_capacity(path)

            max_flow += min_capacity
            found_paths += 1
            augment(path, min_capacity)
        end
    end
    return (max_flow, found_paths)
end

```

• Shortest Augmenting Path

```

function shortest_augmenting_path(vertices::Vector{Vertex}, start::Vertex,
finish::Vertex)::Tuple{Int64, Int64}
    max_flow = 0
    found_paths = 0
    distance_count = distance_label(vertices, finish)
    max_distance = length(vertices)
    pred = Dict{Vertex, Vertex}()
    i = start

    while start.d < length(vertices)
        j = has_admissible_arc(i)
        if j != nothing
            pred[j] = i
            i = j
            if i == finish
                path = reconstruct_path(pred, start, finish)
                min_capacity = find_min_capacity(path)
                augment(path, min_capacity)
                max_flow += min_capacity
                found_paths += 1

                i = start
                pred = Dict{Vertex, Vertex}()
            end
        else
            old_distance = i.d
            retreat(i)
            new_distance = min(i.d, max_distance)

            distance_count[old_distance + 1] -= 1

```

```

        distance_count[new_distance + 1] += 1

        if distance_count[old_distance + 1] == 0 && old_distance < length(vertices)
            break
        end

        if i != start
            temp = pred[i]
            delete!(pred, i)
            i = temp
        end
    end
end

return (max_flow, found_paths)
end

```

- Ustalanie odległości wierzchołków od źródła

```

function distance_label(vertices::Vector{Vertex}, sink::Vertex)
    max_distance = length(vertices)
    distance_count = zeros{Int64, max_distance + 1}

    for v in vertices
        v.d = typemax{Int64}()
    end
    sink.d = 0
    queue = [sink]
    while !isempty(queue)
        current = popfirst!(queue)
        for (neighbor, capacity) in current.neighbours
            if capacity == 0
                if neighbor.d == typemax{Int64}()
                    neighbor.d = current.d + 1
                    distance_count[neighbor.d + 1] += 1
                    push!(queue, neighbor)
                end
            end
        end
    end

    return distance_count
end

```

- Znajdowanie dopuszczalnego łuku

```

function has_admissible_arc(v::Vertex)::Union{Vertex, Nothing}
    for i in (v.last_visited + 1):length(v.neighbours)
        if v.neighbours[i][2] > 0 && v.neighbours[i][1].d == v.d - 1
            last_visited = i
            return v.neighbours[i][1]
        end
    end
    return nothing
end

```

- Retreat

```

function retreat(v::Vertex)
    min_d = typemax{Int64} - 1

    for n in v.neighbours
        if n[2] > 0
            min_d = min(min_d, n[1].d)
        end
    end

    if min_d == typemax{Int64}
        v.d = typemax{Int64}
    else
        v.d = min_d + 1
    end
end

```

- Augment

```

function augment(path::Vector{Vertex}, min_capacity::Int64)
    for i in 1:(length(path) - 1)
        v, w = path[i], path[i + 1]
        index_w = find_neighbour(v, w)
        v.neighbours[index_w] = (v.neighbours[index_w][1],
            v.neighbours[index_w][2] - min_capacity, v.neighbours[index_w][3] + min_capacity)
        index = find_neighbour(w, v)
        w.neighbours[index] = (w.neighbours[index][1],
            w.neighbours[index][2] + min_capacity, w.neighbours[index][3] - min_capacity)
    end
end

```

Implementacja - największe skojarzenie

- Graf

```
mutable struct Vertex
```

```

    value::Int64
    neighbours::Vector{Tuple{Vertex, Int64, Bool}}
    labeled::Bool
end

```

```

mutable struct Graph_struct
    V1::Vector{Vertex}
    V2::Vector{Vertex}
    k::Int64
end

```

- Dodawanie krawędzi

```

function add_edges(graph::Graph_struct, i::Int64)
    for v in graph.V1
        S = sample(graph.V2, i, replace = false)

        for w in S
            push!(v.neighbours, (w, 1, false))
            push!(w.neighbours, (v, 0, false))
            @assert w.value in (2^(graph.k) + 1):(2^((graph.k)+1))
        end
    end
end

```

- Największe skojarzenie

```

function find_biggest_matching(filename::Union{String, Nothing}, graph::Graph_struct,
degree::Int64, print_matching::Bool)::Int64
    s = Vertex(Int64(0))
    t = Vertex(Int64(2^(graph.k+1) + 1))

    for v in graph.V1
        push!(s.neighbours, (v, 1, false))
        push!(v.neighbours, (s, 0, false))
    end

    for w in graph.V2
        push!(t.neighbours, (w, 0, false))
        push!(w.neighbours, (t, 1, false))
    end

    if filename != nothing
        save_to_model(filename, graph, s.neighbours, graph.k, degree, print_matching)
    end

    matching_size = 0

```

```

t.labeled = true

while t.labeled

    list = Queue{Vertex}()
    pred = Dict{Vertex, Vertex}()

    for v in graph.V1
        v.labeled = false
    end

    for v in graph.V2
        v.labeled = false
    end

    t.labeled = false

    enqueue!(list, s)
    s.labeled = true

    while !isempty(list) && !t.labeled
        v = dequeue!(list)
        for (w, capacity, _) in v.neighbours
            if capacity > 0 && !w.labeled
                w.labeled = true
                pred[w] = v
                enqueue!(list, w)
            end
        end
    end

    if t.labeled
        path = reconstruct_path(pred, s, t)
        augment(path)
        matching_size += 1
    end
end

@assert matching_size <= 2^(graph.k)

return matching_size
end

```

Implementacja - programowanie liniowe

Projekt został wzbogacony o możliwość generowania pliku z modelem programowania liniowego w języku Julia z wykorzystaniem JuMP oraz GLPK. W osobnym pliku tworzona jest również struktura reprezentująca krawędzie grafu, na podstawie której odtwarzany jest później graf wejściowy w trakcie działania modelu. Przykładowy plik dla $k = 3$ wygląda następująco:

- Generowanie krawędzi

```
function read_edges(filename::String)::Dict{Tuple{Int64, Int64}, Int64}
    edges = Dict{Tuple{Int64, Int64}, Int64}()
    open(filename) do file
        for line in eachline(file)
            v, w, capacity = split(line)
            edges[(parse(Int64, v), parse(Int64, w))] = parse(Int64, capacity)
        end
    end
    return edges
end
```

- Model

```
k = 3
vertices = [i for i in 0:(2^k - 1)]
edges = read_edges("edges.txt")

start = vertices[1]
finish = vertices[2^k]

model = Model(GLPK.Optimizer)
@variable(model, flow[e in keys(edges)] >= 0)
for e in keys(edges)
    @constraint(model, flow[e] <= edges[e])
end

for v in vertices
    if v != start && v != finish
        @constraint(model, sum(flow[(u, v)] for u in vertices if (u, v) in keys(edges))
            == sum(flow[(v, w)] for w in vertices if (v, w) in keys(edges)))
    end
end

@objective(model, Max, sum(flow[(start, v)] for v in vertices
    if (start, v) in keys(edges)))
```

```
time_max_flow = @elapsed begin
    optimize!(model)
end
```

Testowanie zaimplementowanych algorytmów

Algorytm Edmondsa - Karpa

Algorytm Edmondsa-Karpa został przetestowany na hiperkostkach o wymiarach k z przedziału $\{1, \dots, 16\}$. Dla każdej wartości k została 10-krotnie wygenerowana odpowiednia skierowana hiperkostka H_k , a następnie został obliczony maksymalny przepływ między źródłem $s = 0$ a ujściem $t = 2^k - 1$.

Parametry Wejściowe Programu

1. **-size k**: Wymiar hiperkostki.
2. **-printFlow**: (Opcjonalny) Jeżeli podany, program dodatkowo zwróci przepływ po każdym łuku w sieci.

Efekty Działanie Programu

Program wypisuje na standardowe wyjście:

- Wartość maksymalnego przepływu.
- (Opcjonalnie) Przepływ po każdym łuku.

Na standardowe wyjście błędów program wypisuje:

- Czas działania całego programu.
- Liczbę ścieżek powiększających wyliczanych przez program w trakcie działania algorytmu.

Algorytm Shortest Augmenting Path

Algorytm Shortest Augmenting Path został przetestowany na tych samych danych wejściowych co algorytm Edmondsa-Karpa.

Wyznaczanie największego skojarzenia

Algorytm wyznaczania wielkości największego skojarzenia został przetestowany na grafach o V_1 i V_2 wielkości 2^k , gdzie $k \in \{1, \dots, 16\}$. Dla każdej wartości k została 10-krotnie wygenerowany odpowiedni graf o stopniu wierzchołków $degree \in \{1, \dots, k\}$, dla których zostali niezależnie wybrani sąsiedzi zgodnie z rozkładem normalnym. Następnie został obliczony maksymalny przepływ między źródłem $s = 0$ a ujściem $t = 2^{k+1} + 1$.

Parametry Wejściowe Programu

1. **-size k**: Parametr wyznaczający rozmiary składowych.
2. **-degree i**: Stopień wierzchołków z komponenty V_1
3. **-printMatching**: (Opcjonalny) Jeżeli podany, program dodatkowo zwróci znalezione skojarzenie.

Efekty Działanie Programu

Program wypisuje na standardowe wyjście:

- Wielkość największego skojarzenia.
- (Opcjonalnie) Znalezione skojarzenie.

Na standardowe wyjście błędów program wypisuje:

- Czas działania całego programu.

Model Programowania Liniowego

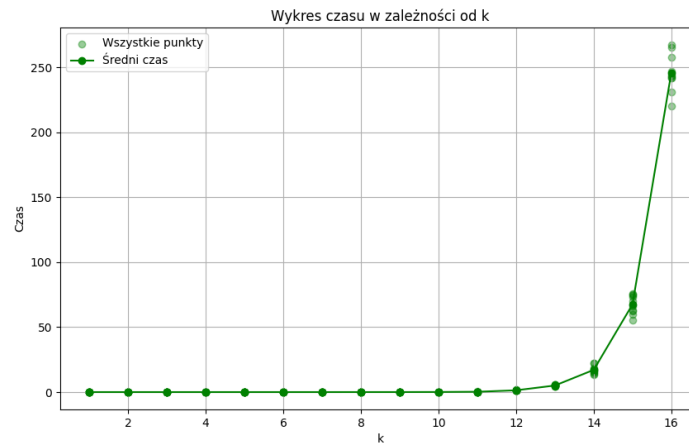
Dla $k \in \{1, \dots, 16\}$ została 10-krotnie wygenerowana hiperkostka wymiaru k . Następnie zostały na niej przetestowane algorytmy Edmondsa-Karpa oraz Shortest Augmenting Path. Graf zapisano do pliku `edges.txt`, a model do pliku `lp.jl`. Następnie uzyskane wyniki w zaimplementowanych algorytmach zostały porównane z wynikami uzyskanymi przez model programowania liniowego. Porównywane były czas działania programu, znaleziony maksymalny przepływ oraz wyznaczony przepływ na każdej ze ścieżek.

Dodatkowo dla $k \in \{3, \dots, 10\}$ oraz $degree \in \{1, \dots, k\}$ został wyznaczony odpowiedni graf dwudzielny, a poprawność wyników uzyskanych w efekcie działania autorskiego algorytmu została porównana z wynikami uzyskanymi przez wygenerowany model programowania liniowego.

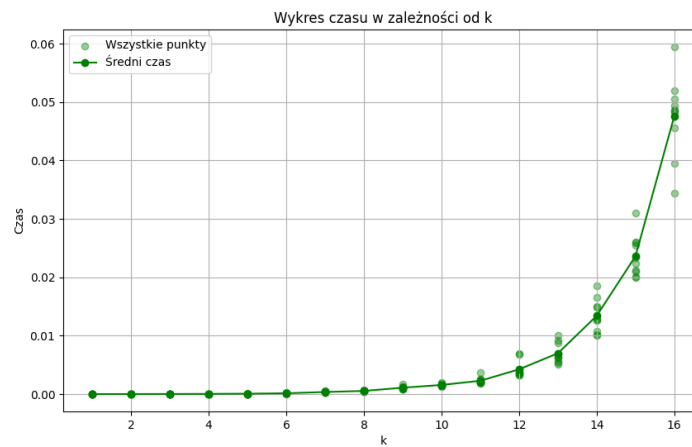
Wyniki

Największy przepływ

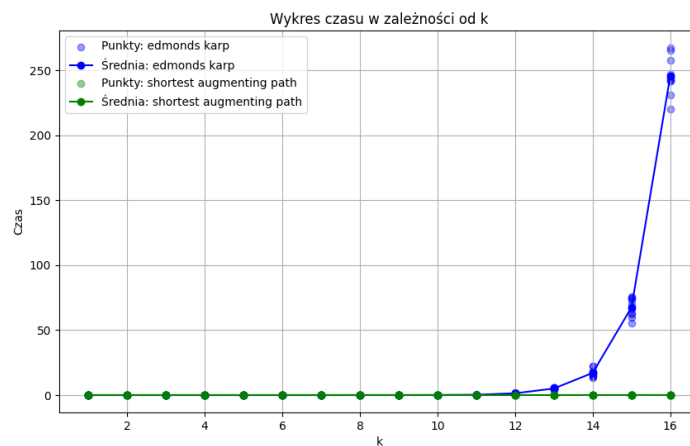
Czas działania



Rysunek 3: Algorytm Edmondsa - Karpa



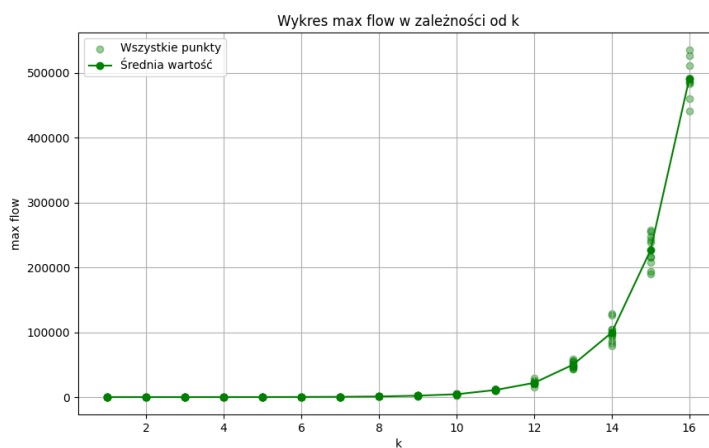
Rysunek 4: Algorytm Shortest Augmenting Path



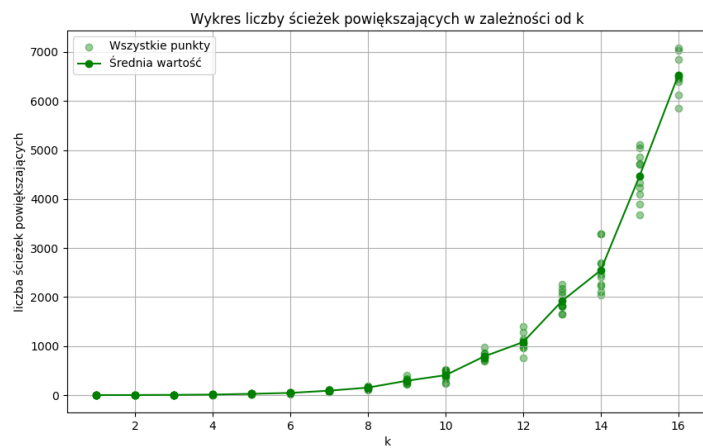
Rysunek 5: Prównanie algorytmów

Parametry maskymalnego przepływu

Algorytmy Edmondsa - Karpa oraz Shortest Augmenting Path zwróciły te same wielkości maksymalnego przepływu oraz liczbę znalezionych ścieżek powiększających.



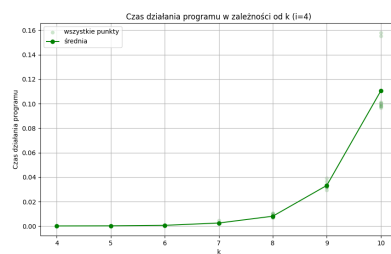
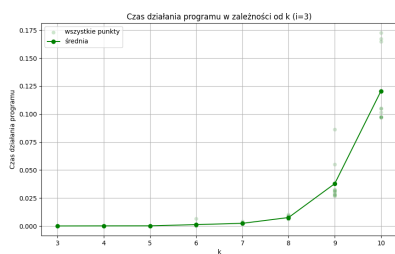
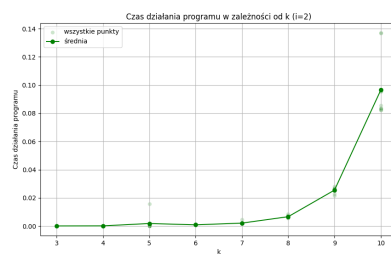
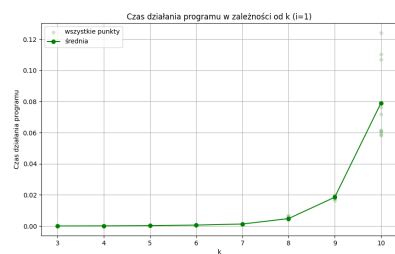
Rysunek 6: Maksymalny przepływ

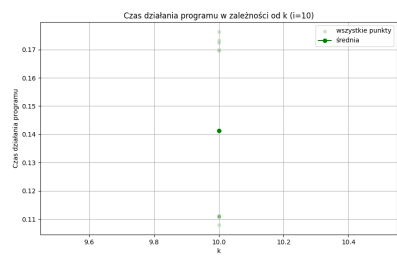
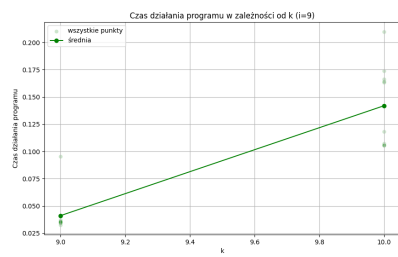
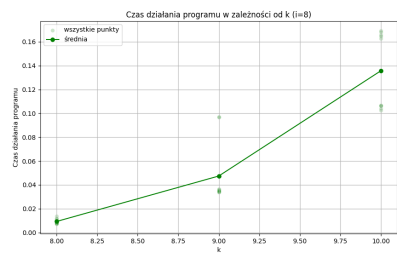
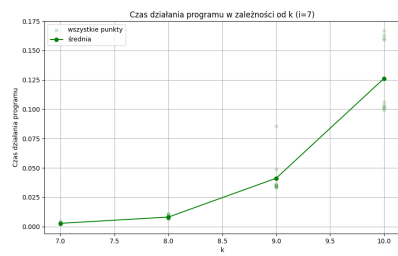
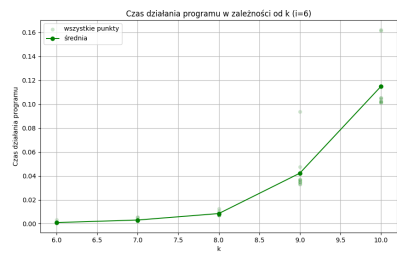
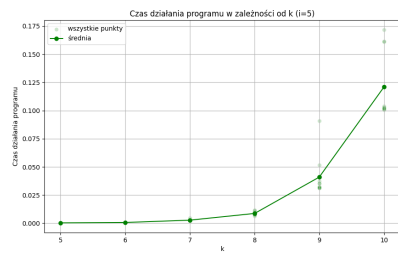


Rysunek 7: Liczba ścieżek powiększających

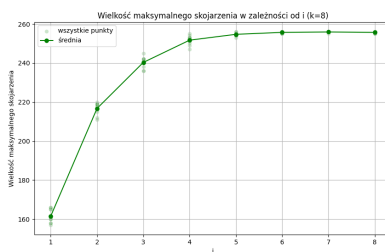
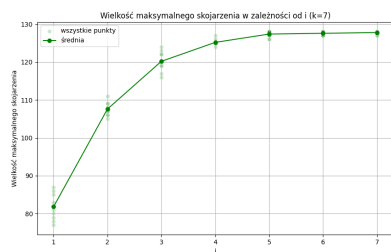
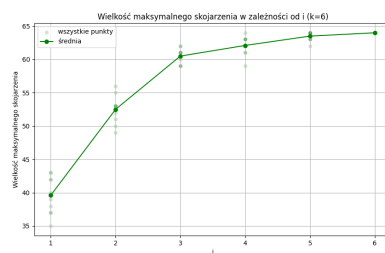
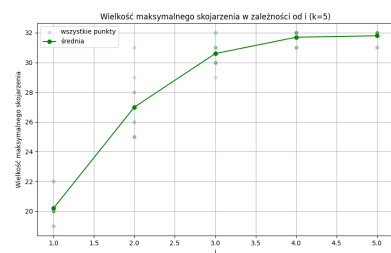
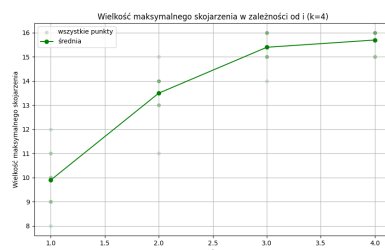
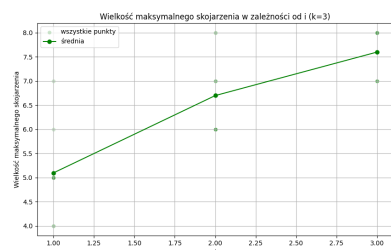
Największe skojarzenie

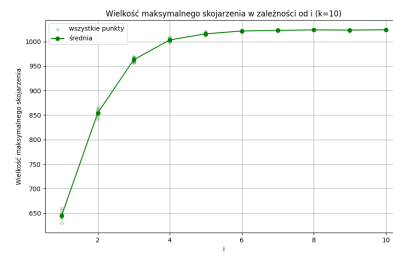
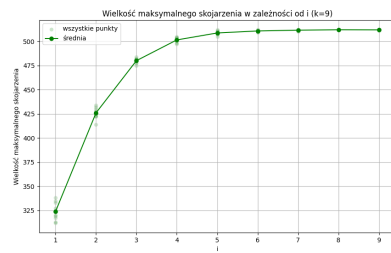
Czas działania





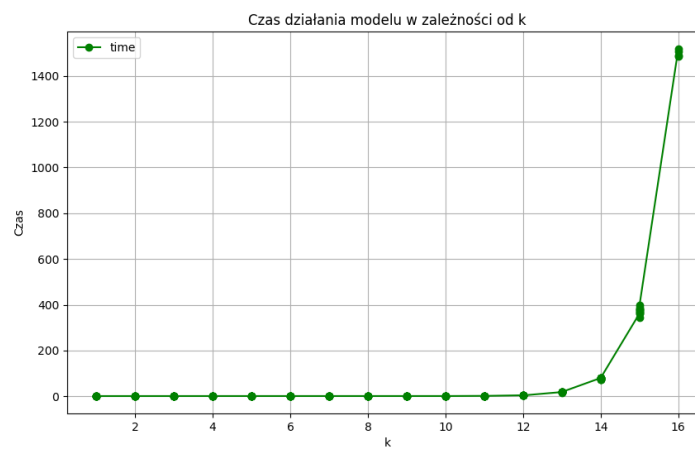
Wielkość skojarzenia



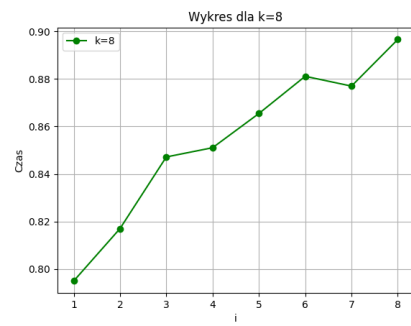
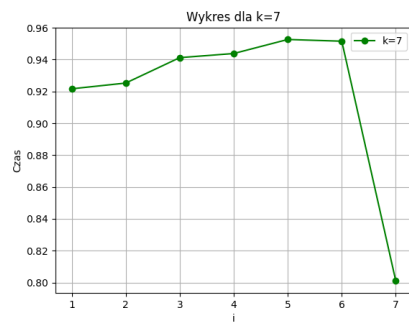
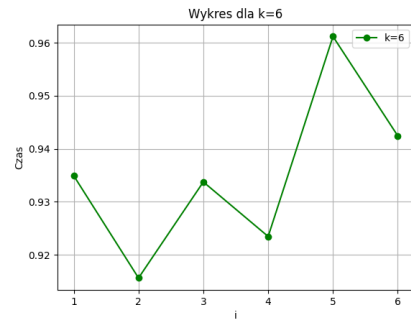
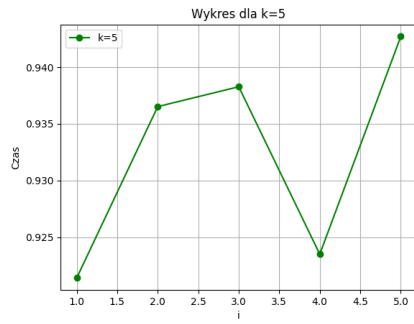
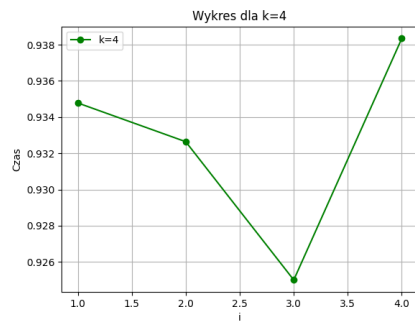


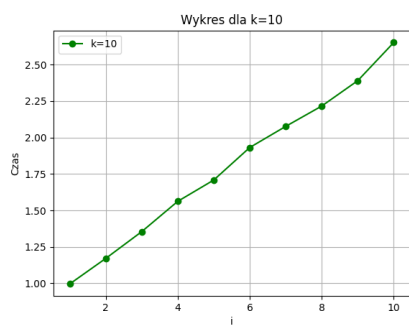
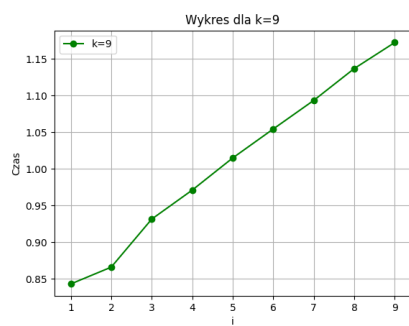
Model programowania liniowego

Czas działania



Rysunek 17: Czas działania modelu liniowego (maksymalny przepływ)





Rysunek 21: Czas działania modelu (największe skojarzenie)

Porównanie wielkości maksymalnego przepływu

k	max flow	łuk	EK	SAP	LP
2	4	0 -> 1	3	3	3
		0 -> 2	1	1	1
		1 -> 3	3	3	3
		2 -> 3	1	1	1
3	5	0 -> 1	1	1	1
		0 -> 2	3	3	3
		0 -> 4	1	1	1
		1 -> 3	0	0	0
		1 -> 5	1	1	1
		2 -> 3	2	2	2
		2 -> 6	1	1	1
		3 -> 7	2	2	2
		4 -> 5	1	1	1
		4 -> 6	0	0	0
		5 -> 7	2	2	2
		6 -> 7	1	1	1
4	23	0 -> 1	6	6	6
		0 -> 2	12	12	12
		0 -> 4	3	3	3
		0 -> 8	2	2	2
		1 -> 3	2	2	2
		1 -> 5	2	2	2
		1 -> 9	2	2	4
		2 -> 3	6	6	6
		2 -> 6	2	2	2
		2 -> 10	4	4	4
		3 -> 7	1	1	1
		3 -> 11	7	7	7
		4 -> 12	3	3	3
		5 -> 13	2	2	2
		6 -> 14	2	2	2
		7 -> 15	1	1	1
		8 -> 10	2	2	2
		9 -> 11	2	2	2
		10 -> 11	4	4	4
		10 -> 14	2	2	2
		11 -> 15	13	13	13
		12 -> 14	3	3	3
		13 -> 15	2	2	2
		14 -> 15	7	7	7

Tabela 1: Porównanie wielkości maksymalnego przepływu

Porównanie wielkości największego skojarzenia

k	matching	BPM	LP
2	4	1->8 2-> 7 3->6 4 -> 5	4->7 1->5 2->8 3 -> 6
3	6	1 -> 10 2 -> 16 3 -> 9 4 -> 13 7 -> 14 8 -> 15	6->10 8->15 5->9 7->14 4->13 2->16
4	13	1 -> 25 2 -> 32 3 -> 26 4 -> 21 6 -> 17 7 -> 28 8 -> 30 9 -> 27 10 -> 20 11 -> 19 12 -> 18 13 -> 31 14 -> 23	11->17 14->23 1->31 4->21 9->27 2->32 15->19 16->25 3->26 7->28 10->29 8->30 12->18

Tabela 2: Porównanie wielkości największego skojarzenia

Wnioski

Maksymalny przepływ

Porównując trzy algorytmy: Edmondsa - Karpa, Shortest Augmenting Path oraz wykorzystując programowanie liniowe, tym, który radzi sobie najlepiej jest algorytm Shortest Augmenting Path. Czas działania programu dla największych danych 2^{16} wierzchołków oraz 2^{19} krawędzi jest rzędu setnych części sekundy, co jest wynikiem bardzo dobrym w porównaniu do algorytmu Edmondsa - Karpa, któremu zajmuje to kilkaset sekund czy algorytmu modelu programowania liniowego, dla którego czas działania wynosi kilka tysięcy sekund.

Dane uzyskane w algorytmach Edmondsa - Karpa oraz Shortest Augmenting Path zgadzają się z tymi uzyskanymi w modelu programowania liniowego, co potwierdza ich poprawność.

Wykresy potwierdziły teoretyczne złożoności algorytmów. Która w tym przypadku wynosi $O(k^2 2^{3k-2})$ dla algorytmu Edmondsa - Karp'a oraz $O(k 2^{3k-1})$ dla algorytmu Shortest Augmenting Path.

Największe skojarzenie

Algorytm wyznaczania największego skojarzenia w opraciu o algorytm max flow okazał się być efektywnym sposobem wyznaczania największego skojarzenia w zaproponowanym grafie dwudzielnym.

Wyniki uzyskane przez algorytm pokrywają się z tymi uzyskanymi przez model programowania liniowego, z dokładnością do wyboru krawędzi do skojarzenia, co wskazuje jedynie na różnicę w działaniu algorytmów, a nie błędną implementację jednego z nich.

Wykresy potwierdziły teoretyczną złożoność algorytmu, która dla tego przypadku wynosi $O(i 2^{k+2})$

Podsumowanie

Zaimplementowane algorytmy stanowią znaczne przyspieszenie w stosunku do modelu programowania liniowego, jednocześnie zwracając poprawne wyniki, co świadczy o sukcesie projektu.

Literatura

- [1] T.L. Magnanti R.K. Ahuja and J.B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Pearson Education, 2014.